Proyecto final Ingeniería de Software V: Circle Guard

Integrantes:
- Melissa Hurtado Becerra - A00401116
- Isabella Cuervo Vargas - A00401334
- Valentina Tobar Gomez - A00401749
- Santiago Grajales Perez - A00402018

Sprint 0

Contexto del proyecto:
CircleGuard es un sistema de contact tracing y cercado sanitario para un campus universitario. Su propósito es identificar grupos de contacto interconectados, denominados Circles, y propagar estados de riesgo de forma recursiva cuando una persona resulta positiva.

El sistema busca contribuir a la contención temprana de brotes, sin exponer la identidad real de los estudiantes. Para ello, el grafo de contactos utiliza identificadores anónimos, mientras que la información de identidad se mantiene aislada en un componente denominado Identity Vault.

Flujo actual:

- Un usuario reporta síntomas o es marcado como positivo por el Centro de Salud.
- El sistema propaga el riesgo a sus contactos mediante el grafo Neo4j, utilizando las relaciones ENCOUNTERED y MEMBER_OF dentro de una ventana temporal.
- Los estados de riesgo se propagan de forma recursiva: CONFIRMED → SUSPECT → PROBABLE.
- Los usuarios afectados reciben notificaciones por diferentes canales.
- El sistema permite la liberación automática al finalizar la ventana de aislamiento.
- El acceso al campus se valida mediante códigos QR, utilizando Redis para consultar el estado de los usuarios.

Objetivo del proyecto:
Implementar prácticas DevOps en CircleGuard, asumiendo la responsabilidad de llevar el software desde el commit de un desarrollador hasta producción mediante un proceso automatizado, seguro, observable y reproducible, teniendo en cuenta las métricas de éxito del producto.

Metodología:
El proyecto tendrá una duración total de 25 días, distribuidos en tres sprints. Se utilizará Scrum como marco de trabajo para organizar las actividades, gestionar el trabajo pendiente y realizar seguimiento periódico al progreso del equipo.

- **Sprint 0 – Preparación:** conformación del equipo, definición del marco ágil, establecimiento de acuerdos de trabajo, estrategia de branching y definición del backlog inicial.

- **Sprint 1 – Integración continua y entrega continua:** implementación de un pipeline CI/CD inicial que funcione de punta a punta. Se documentará el Sprint Review y el Sprint Retrospective.

- **Sprint 2 – Infraestructura y madurez DevOps:** implementación de Terraform, Kubernetes, observabilidad, patrones, seguridad y documentación. Se documentará el Sprint Review, Sprint Retrospective y las lecciones aprendidas.

Desarrollo sprint 0: El Sprint 0 tuvo una duración de un día y estuvo enfocado en establecer las condiciones organizativas y técnicas necesarias para iniciar el trabajo del equipo.

Sus actividades se documentan en este apartado y no se gestionan como historias dentro del backlog de Jira, dado que corresponden a decisiones y acuerdos iniciales de organización, previos a la planificación del trabajo de desarrollo. Las tareas realizadas fueron:

- Conformar el equipo
- Definición del marco ágil
- Definición de estrategia de branching
- Definición del backlog
- Analisis del codigo base

A continuación se documentan las tareas realizadas:

1. **Conformación del equipo:** El equipo se encuentra conformado por Valentina Tobar Gomez, Isabella Cuervo Vargas, Melissa Hurtado Becerra y Santiago Grajales Perez.

   La conformación del equipo se fundamenta en la experiencia previa de trabajo conjunto y en la cohesión desarrollada durante los semestres anteriores. Esta experiencia ha permitido establecer una dinámica de colaboración caracterizada por una comunicación fluida, conocimiento de las fortalezas de cada integrante y capacidad para trabajar de manera coordinada.

2. **Definición del marco ágil:** Como equipo se decidió utilizar Scrum como marco de trabajo, principalmente por la experiencia previa de sus integrantes y el conocimiento adquirido durante su aplicación en proyectos académicos anteriores.

   Esta familiaridad facilita la adopción del marco y permite aprovechar prácticas conocidas para organizar el trabajo, priorizar actividades y realizar seguimiento al progreso. Para la organización del equipo se contemplan las siguientes responsabilidades:

   - **Product Owner:** Melissa Hurtado Becerra
   - **Scrum Master:** Santiago Grajales Perez

- **Equipo de desarrollo:** los cuatro integrantes del equipo participarán en las actividades de desarrollo y construcción de los entregables.

3. **Definición de estrategia de branching:** Como equipo decidimos adoptar la estrategia de gestión de ramas establecida por la universidad en el DevOps Sandbox. Esta decisión se fundamenta en que ya hemos trabajado con esta estrategia en asignaturas anteriores, como Proyecto Integrador II y Proyecto de Grado, por lo que contamos con experiencia previa en su aplicación y conocemos su dinámica de trabajo. La estrategia corresponde a una adaptación simplificada de GitFlow, en la que se mantienen las ramas permanentes main y dev, y se crean ramas independientes para el desarrollo de cada historia de usuario.

   La estrategia busca mantener un flujo organizado de desarrollo, facilitar la colaboración entre los integrantes y garantizar que los cambios sean revisados antes de integrarse al código principal.

   - **Ramas principales**
     main: rama principal del proyecto. Contendrá el código integrado, revisado y aprobado para su despliegue en producción. Estará protegida y su integración se realizará mediante Pull Requests aprobados por los Maintainers.

     dev: rama de integración del equipo de desarrollo. Recibirá los cambios correspondientes a las diferentes historias de usuario una vez hayan sido revisados y aprobados. También estará protegida para evitar modificaciones directas.

   - **Creación de ramas de trabajo**
     Por cada historia de usuario se creará una rama independiente a partir de dev. Se utilizará la siguiente convención:

     hu-[identificador]-[resumen]

     Ejemplos:
     hu-01-user-authentication
     hu-02-create-ci-pipeline
     hu-03-configure-docker

     Los nombres deberán escribirse en minúsculas, utilizando guiones para separar las palabras y evitando espacios o caracteres especiales.Una vez integrada la historia de usuario, la rama podrá eliminarse.

   - **Gestión de Pull Requests**
     Cuando un desarrollador finalice una historia de usuario, deberá crear un Pull Request hacia la rama dev para solicitar la revisión e integración de sus cambios.

     El título seguirá la siguiente estructura:

HU-[identificador]: [tipo]: [descripción breve]

Ejemplo:
HU-02: feat: implement initial ci pipeline

La descripción del Pull Request deberá contener:

Context: explicación del problema o necesidad que aborda la historia.
Changes: descripción de los cambios implementados.
How to test: instrucciones para validar los cambios y los resultados esperados.

Todo Pull Request deberá contar con la aprobación de al menos un integrante diferente de su autor. Los comentarios de revisión y posibles conflictos deberán resolverse antes de realizar la integración.

Para promover los cambios hacia main, se deberá crear un Pull Request desde dev, sujeto a revisión y aprobación. Una historia de usuario se considerará integrada cuando sus cambios hayan sido aceptados e incorporados a dev, de acuerdo con los criterios de aceptación definidos.

- **Protección de ramas**
  Las ramas main y dev contarán con reglas de protección para mantener la integridad del código. Se establecerán las siguientes restricciones:

  - No permitir modificaciones directas mediante push.
  - Exigir Pull Requests para integrar cambios.
  - Exigir al menos una aprobación de otro integrante.
  - Prohibir el force push.
  - Prohibir la eliminación de las ramas principales.
  - Exigir que las verificaciones automatizadas del pipeline sean satisfactorias una vez estén configuradas.
  - La administración de estas reglas estará a cargo de los integrantes del repositorio.

- **Buenas prácticas de colaboración**
  Idioma: los nombres de las ramas, los mensajes de commits, los títulos y las descripciones de Pull Requests se escribirán completamente en inglés.

  Estructura de commits: se utilizará una convención basada en tipos de cambio, manteniendo los mensajes en minúsculas y con descripciones claras.

  - Nueva funcionalidad: feat: add [feature] - [description]
  - Corrección de errores: fix: correct [issue] - [description]
  - Mejora de funcionalidad: enhance: improve [feature] - [description]
  - Nueva documentación: docs: add [document] - [description]

- Actualización de documentación: docs: update [document] - [description]
- Nuevas pruebas: test: add [test] - [description]

Los mensajes deberán ser concretos y describir el cambio realizado.

Versionado semántico y generación de releases: Se adoptará Conventional Commits como estándar para los mensajes de commit. Esta convención permitirá identificar la naturaleza de los cambios realizados y servirá como base para automatizar el versionado semántico del proyecto y la generación de release notes.

Los incrementos de versión se determinarán de acuerdo con las siguientes reglas:

- **MINOR:** cuando se incorporen nuevas funcionalidades mediante commits feat.
  Ejemplo:
  feat(promotion): add status cascade with 14-day TTL

- **PATCH:** cuando se realicen correcciones de errores mediante commits fix.
  Ejemplo:
  fix(gateway): correct signed QR expiration

- **MAJOR:** cuando se introduzcan cambios incompatibles (breaking changes), identificados mediante ! en el tipo de commit o BREAKING CHANGE: en el cuerpo o pie del mensaje.
  Ejemplo:
  feat(auth)!: replace role management schema

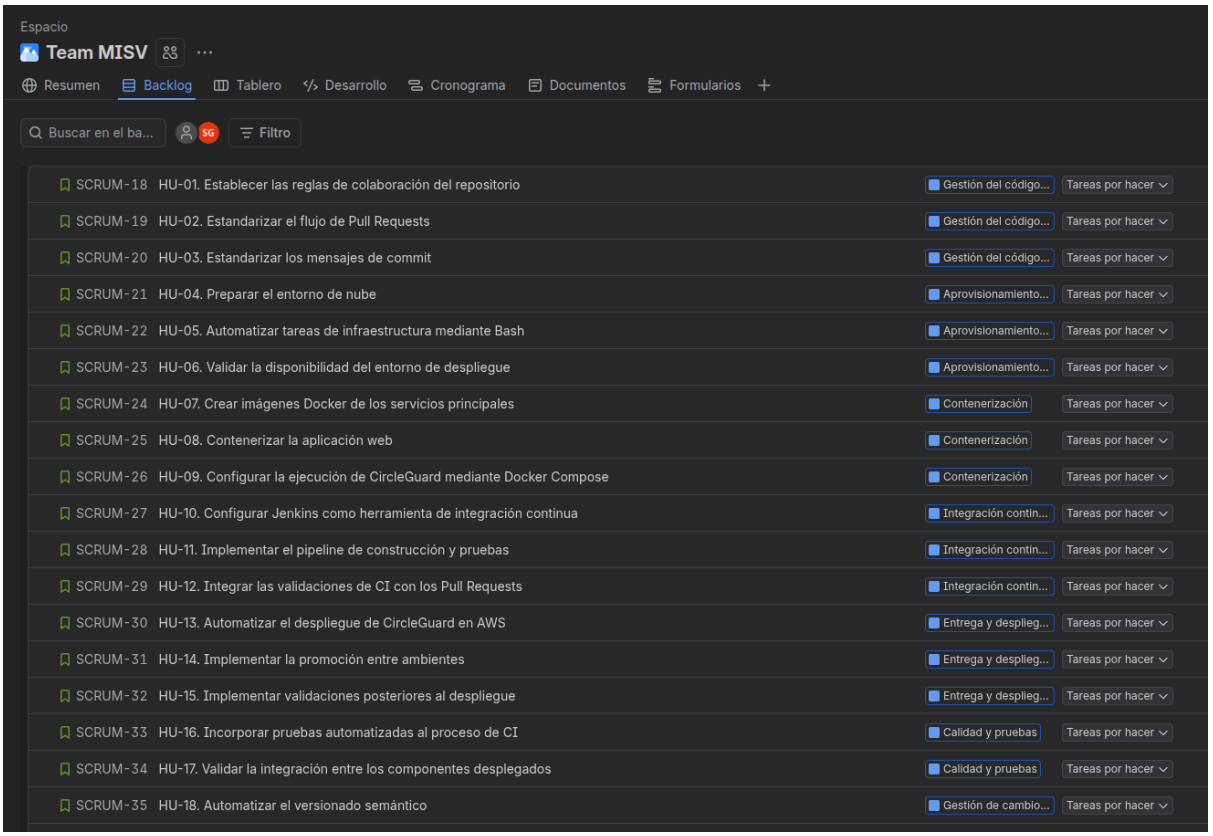
La automatización del versionado semántico y la generación de release notes se realizará mediante semantic-release integrado con GitHub Actions. Esta herramienta analizará los commits incorporados a la rama de publicación, determinará el incremento de versión correspondiente y generará las notas de la nueva release.

4. **Definición del backlog:**Como parte de la preparación del proyecto, se realizó una identificación inicial del trabajo necesario para implementar las prácticas DevOps en CircleGuard.

   Como resultado, se definieron 13 épicas, que agrupan los principales objetivos y áreas de trabajo del proyecto, y 32 historias de usuario, que describen los entregables necesarios para alcanzar dichos objetivos.
   Cada historia de usuario cuenta inicialmente con una descripción, criterios de aceptación y prioridad, proporcionando una visión general del trabajo requerido y facilitando la planificación de los siguientes sprints.El backlog completo puede consultarse en el siguiente enlace:

Link de backlog:
https://circleguard-msvi.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog

La gestión y seguimiento del backlog se realizará mediante Jira, herramienta en la que se administrarán las historias de usuario, sus prioridades, asignaciones y estados de avance durante el desarrollo del proyecto.



5. **Análisis del código base:** Como parte de la preparación del proyecto, se realizó una revisión del repositorio de CircleGuard con el propósito de comprender su arquitectura, identificar sus principales componentes y evaluar el estado inicial de la aplicación antes de incorporar las prácticas DevOps.

   El análisis permitió identificar diferencias entre la documentación del proyecto y su implementación actual. Entre ellas se encontraron discrepancias en la versión de Spring Boot, la ausencia de documentación OpenAPI y de artefactos de Kubernetes, así como diferencias entre los mecanismos de almacenamiento descritos y los implementados. También se identificó que algunas pruebas de integración con Testcontainers están concentradas en determinados servicios.
   Adicionalmente, se encontraron aspectos que representan riesgos para la seguridad, la estabilidad y el mantenimiento del sistema. Entre los más relevantes se encuentran controles de autorización incompletos en algunos servicios, credenciales y secretos almacenados directamente en archivos del repositorio, inconsistencias en

la configuración de la expiración de los códigos QR y configuraciones de serialización Kafka que podían provocar fallos durante la publicación de eventos.

En la aplicación móvil también se identificaron errores relacionados con la validación de códigos QR, la gestión de sesiones y la sincronización de datos de proximidad. Asimismo, la revisión permitió reconocer componentes parcialmente implementados, servicios simulados (mocks), métodos sin uso y funcionalidades que actualmente no cuentan con una implementación completa.

Como parte de esta revisión, se realizaron correcciones a problemas que afectaban la ejecución y validación del proyecto. Entre ellas se encuentran la resolución de un conflicto de caché que provocaba errores en el flujo de propagación de estados, el ajuste de la configuración de productores Kafka, la actualización de Testcontainers para garantizar su compatibilidad con la versión de Docker utilizada, la corrección de autoridades en pruebas de seguridad y el ajuste de propiedades requeridas por las pruebas del servicio de identidad. También se corrigió la gestión de archivos generados que no debían estar versionados en Git.

Finalmente, se validó el funcionamiento del entorno de desarrollo y de los principales flujos de la aplicación. Como resultado de las correcciones, la suite de pruebas ejecutada finalizó con 52 pruebas exitosas y 0 fallos, los ocho microservicios lograron iniciar y se verificó el flujo de propagación de estados y publicación de eventos mediante Kafka.

Este análisis constituye una línea base técnica para el proyecto. Los hallazgos identificados permiten reconocer las condiciones iniciales del sistema, anticipar posibles dificultades durante la implementación y orientar el trabajo de automatización, pruebas, seguridad, despliegue y observabilidad. Los problemas detectados no se consideran automáticamente parte del alcance de corrección del proyecto; su tratamiento deberá priorizarse de acuerdo con los objetivos y requisitos establecidos para cada parte.

6. **Sprint review:**Durante la revisión del Sprint 0 se evaluaron las actividades de preparación realizadas por el equipo y su contribución al objetivo general del proyecto. Se verificó el cumplimiento de las principales actividades planificadas:

    - **Conformación del equipo:** se estableció el equipo de trabajo y se reconoció la experiencia previa de sus integrantes trabajando conjuntamente.
    - **Definición del marco ágil:** se seleccionó Scrum como marco de trabajo para organizar las actividades y dar seguimiento al proyecto.
    - **Estrategia de branching:** se definió una estrategia de gestión de ramas basada en una adaptación simplificada de GitFlow, estableciendo las reglas para el desarrollo colaborativo y la integración de cambios.
    - **Definición del backlog inicial:** se identificaron 13 épicas y 32 historias de usuario, con sus respectivas descripciones, criterios de aceptación y prioridades iniciales.

- **Análisis del código base:** se revisó la arquitectura y el estado inicial de CircleGuard, identificando inconsistencias, riesgos técnicos y oportunidades de mejora que deberán tenerse en cuenta durante los siguientes sprints.

Como resultado, el equipo estableció una base organizativa y técnica para iniciar la implementación de las prácticas DevOps. Las actividades relacionadas con el aprovisionamiento de infraestructura y la configuración de herramientas se trasladaron al Sprint 1, al corresponder a tareas de implementación técnica.

En conclusión, el Sprint 0 cumplió su propósito de preparar al equipo, definir la forma de trabajo y establecer una visión inicial del trabajo requerido para el proyecto.

7. **Sprint Retrospective:** Al finalizar el Sprint 0, el equipo realizó una reflexión sobre la manera en que se desarrollaron las actividades de preparación, identificando los aspectos positivos y aquellos que pueden mejorarse durante los siguientes sprints.

   ¿Qué hicimos bien?

   - Aprovechamos la experiencia previa de trabajo conjunto, lo que facilitó la comunicación y la coordinación entre los integrantes.
   - Definimos una metodología de trabajo conocida por el equipo, facilitando la organización inicial de las actividades.
   - Establecimos una estrategia de branching y convenciones de colaboración para mantener un flujo de trabajo ordenado.
   - Realizamos un análisis del código base antes de comenzar la implementación, permitiendo conocer mejor el estado del proyecto y anticipar posibles dificultades.
   - Construimos un backlog inicial que proporciona una visión general del trabajo necesario para alcanzar los objetivos del proyecto.

   ¿Qué podemos mejorar?

   - Profundizar en la revisión de los requisitos para asegurar que las historias de usuario y sus criterios de aceptación cubran completamente el alcance solicitado.
   - Mejorar la estimación y priorización de las actividades para distribuir el trabajo de manera equilibrada.
   - Evitar asumir que las funcionalidades descritas en la documentación del proyecto están implementadas; es necesario contrastar los requisitos con el comportamiento real del código.
   - Mantener actualizada la documentación a medida que se tomen decisiones y se realicen cambios técnicos.

   Acciones de mejora para el siguiente sprint

   - Revisar y ajustar el backlog antes de iniciar la implementación, teniendo en cuenta los requisitos y criterios de evaluación del proyecto.

- Mantener la estrategia de branching y las convenciones establecidas durante el desarrollo.
- Registrar los avances, decisiones técnicas y dificultades encontradas para facilitar el seguimiento del proyecto.